A

SEMINAR REPORT

ON

WEB SCRAPING

BACHELOR OF TECHNOLOGY

In

Information Technology

Submitted by:
Muskan Atray
(Roll No: **2216853)**

Supervised by:
Dr. Kusum Gupta

AIM AND ACT DEPARTMENT

BANASTHALI VIDYAPITH,

RAJASTHAN

AUGUST 2024

ABSTRACT

This report delves into the concept and application of web scraping, a powerful method used to extract data from websites for a variety of purposes, including market research, sentiment analysis, and pricing strategies. It compares web scraping with screen scraping, highlighting their differences and specific use cases. The report also addresses the legal implications, ethical concerns, and technical challenges associated with web scraping, such as changing website structures, CAPTCHA systems, and handling large-scale data extraction. Additionally, it explains key components like web crawlers, HTTP requests, and popular scraping tools. Ultimately, the report emphasizes the significance of web scraping in data-driven decision-making while recognizing its limitations and the need for responsible use.

TABLE OF CONTENT

SERIAL NO.	TITLE	PAGE NO.
1	ABSTRACT	1
2	INTRODUCTION	4
3	SCREEN AND WEB SCRAPING	5
4	IS WEB SCRAPING LEGAL?	6
5	COMPONENTS OF DATA PROCESSING	7
6	WORKING OF WEB SCRAPING	9
7	COMMONLY USED TOOLS	11
8	APPLICATION	12
9	CHALLENGES AND LIMITATIONS	13
10	CONCLUSION	14
11	REFERENCES	15

CHAPTERS

1. **Introduction**
   1.1.Definition of Web Scraping
   1.2.Brief overview of its application
2. **Web scraping and Screen Scraping**
   2.1 Screen Scraping
   2.2 Web Scraping
   2.3 Comparison Of Web and Screen Scraping
3. **Is Web Scraping Legal?**
   3.1. Legality
   3.2.Laws Regarding Web Scraping
   3.3.Major legality issues occurred
4. **Components of Data extraction**
   4.1. Web Crawling
   4.2.Web Scraping
   4.3.HTML Request(GET and POST)
5. **Working Of Web scraping**
6. **Commonly used tools**
   6.1.Scrapy
   6.2.Beautiful Soup
   6.3.ScrapingBee
   6.4.Selenium
7. **Applications**
8. **Challenges and Limitations**
9. **Conclusion**
   9.1.Summary of key points
   9.2.Final thoughts on its Usage
10. **References**

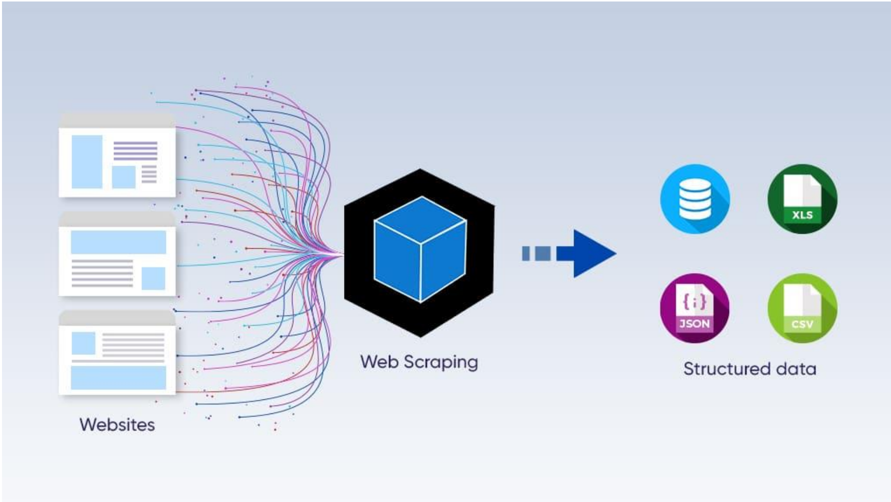
INTRODUCTION

Web scraping, also known as web extraction or harvesting, is a widely used technique for extracting data from the World Wide Web (WWW) and saving it to a file system or database for later retrieval and analysis. This procedure can be carried out manually or automatically using web crawlers or bots. Typically, web scraping uses web browsers or HTTP to gather data from websites.

Web scraping is regarded as an effective and potent technique for gathering and analyzing big data because of the enormous volume of information that is continuously generated on the WWW. It is used in many different areas, including sentiment analysis, pricing monitoring, competitive analysis, and market research. Businesses use web scraping to monitor customer behavior and obtain pricing information from competitors,and stay updated on industry trends.

Web scraping is also very important in industries like e-commerce and finance. It is employed in lead generation, product monitoring, financial data aggregation, and even academic research to collect and analyze big datasets. Web scraping has advantages, but it also has drawbacks, including ethical and legal questions and technological problems such websites' anti-scraping policies or frequent structural changes.

In today's digital age, scraping plays a crucial role in supporting data-driven decision-making by effectively utilizing data from the internet.


Websites
Web Scraping
Structured data
XLS
JSON
CSV

WEB SCRAPING AND SCREEN SCRAPING

- **Screen Scraping** refers to the process of extracting data from the visual output of a web page or software, usually by user interaction. It captures the displayed information on the screen rather than the underlying HTML or code of the webpage.
- **Web Scraping**, on the other hand, is a method to programmatically extract data directly from the HTML code of web pages. It allows for more precise and structured data collection by accessing a web page’s source code through tools and libraries.

Comparison:

1. **Data Source**: Screen scraping focuses on the graphical interface, while web scraping deals with the structured HTML content.
2. **Precision**: Web scraping is more accurate since it directly targets specific elements within the page's source, whereas screen scraping can be more prone to errors due to changes in the visual layout.
3. **Usage**: Web scraping is preferred for collecting large-scale data for analysis, whereas screen scraping is commonly used for older systems or when HTML access is restricted.
4. **Complexity**: Web scraping requires understanding the underlying code structure (like HTML and CSS), while screen scraping mimics user interactions and can work even without knowing the page’s source code.

Both methods are valuable, but web scraping is considered more efficient and versatile for large-scale data extraction across modern web environments.

IS WEB SCRAPING LEGAL?

The legality of web scraping depends on how it's conducted and the purpose behind it. Web scraping is generally legal when done without violating laws or the terms of service (ToS) of the target website. However, malicious actors may use scraping tools to access and misuse data, violating copyright norms or privacy regulations, which can tarnish the industry's reputation.

Overuse of scraping can overload a website's server, resulting in the implementation of security measures against it. Therefore, even if ethical scraping helps obtain data that is publicly available, it is always important to respect legal limits to prevent legal repercussions.

But in case you are looking forward to using it as your own without the consent of the owner and by violating the 'Terms & Conditions' Guidelines, here it will be treated as illegal. However, the law regarding Web Scraping is not transparent but there are still some regulations in which you can fall for doing unauthorized web scraping. Some of these are listed below:

- Violation of the Digital Millennium Copyright Act (DMCA)
- Violation of the Computer Fraud and Abuse Act (CFAA)
- Breach of Contract
- Copyright Infringement
- Trespassing, etc.

Major Legal disputes:

1. ***LinkedIn Vs HiQ***. HiQ is a data analytics firm that came in a legal dispute with LinkedIn when the latter sent an official letter to HiQ demanding it to stop scraping the site. But LinkedIn got a counter-attack from HiQ as they stated that the data of LinkedIn is accessible to anyone who visits it and there is nothing false in scraping the publicly available data. However, the final decision was not praiseworthy by LinkedIn as the court banned the company from blocking HiQ's requests to scrape data from publicly available profiles on the platform. This case has something different as unlike earlier Web Scraping legal disputes, here the court did not favor the company whose data was being scraped.

2. ***Facebook Vs Power Ventures.*** It is a legal action brought by Facebook claiming that Power Ventures Inc. has gathered the user data from Facebook and use it on their website. Facebook alleged that the company had violated the Computer Fraud and Abuse Act (CFAA), and the California Comprehensive Computer Data Access and Fraud Act. As per Facebook, Power Ventures also violated the CAN-SPAM Act by using Facebook's identity while doing the process of extracting user data. In the defense, Power Ventures stated that Facebook's DMCA claim was not sufficient to be considered. They also said that the unauthorized access was not met because the users are actually accessing their own data on Facebook via Power Ventures platform. Although, despite all these arguments, the court's decision came in favor of facebook

COMPONENTS OF DATA PROCESSING

Web Crawlers:

Web crawlers are automated programs that traverse the web to gather pages by following hyperlinks. They begin with an initial URL (seed URL) and continue exploring links from one page to the next. This exploration helps in gathering raw HTML data which will be processed in the next steps. Crawlers are designed to efficiently cover as many links as possible, discovering new pages and adding them to the list of URLs to crawl.

```python
import requests
from bs4 import BeautifulSoup

def crawl(url):
    page = requests.get(url)
    soup = BeautifulSoup(page.content, 'html.parser')
    links = [a['href'] for a in soup.find_all('a', href=True)]
    return links

start_url = "http://example.com"
urls_to_crawl = [start_url]

for url in urls_to_crawl:
    links = crawl(url)
    urls_to_crawl.extend(links)
```

Web Scrapers: Once crawlers have retrieved the required web pages, web scrapers are used to extract the needed data from the HTML structure. This data can be in the form of text, images, or links. Web scrapers use various techniques, such as HTML parsing and regular expressions, to locate and extract relevant data elements. Libraries like BeautifulSoup (Python) make it easier to identify elements using tags, IDs, classes, or attributes.

```python
import requests
from bs4 import BeautifulSoup

def extract_data(url):
    page = requests.get(url)
    soup = BeautifulSoup(page.content, 'html.parser')
    titles = soup.find_all('h1')
    return [title.get_text() for title in titles]

titles = extract_data("http://example.com")
print(titles)
```

HTTP Requests (GET & POST):

GET Request: A GET request is used to retrieve data from a server. When a GET request is made, the server sends back the requested resource (such as an HTML page). This method does not change the server's data, it simply requests it. Web scrapers typically use GET requests to download web pages.

```
response = requests.get("http://example.com")
if response.status_code == 200:
    soup = BeautifulSoup(response.content, 'html.parser')
    print(soup.prettify())
```

POST Request: A POST request, on the other hand, is used to send data to a server, usually to update or submit information (like a form). Web scrapers may use POST requests when interacting with websites that require form submissions (like a login page or data query).

```
data = {"username": "user", "password": "pass"}
response = requests.post("http://example.com/login", data=data)
if response.status_code == 200:
    soup = BeautifulSoup(response.content, 'html.parser')
    print(soup.prettify())
```

WORKING OF WEB SCRAPING

1. Identification of Target URLs

This is the first step where you define which web pages you want to scrape. These URLs contain the data you need to extract from the web. It's important to identify relevant and accessible pages.

2. Check robots.txt and Respect Website Policies

Before scraping, it's essential to check the website's robots.txt file. This file tells you which pages are allowed or disallowed for scraping. By respecting these rules, you avoid violating the website's policies.

3. Choosing the Appropriate Proxy Server (if necessary)

Proxies are often used to hide your scraping activities or to avoid getting blocked by the website. If scraping multiple pages or large amounts of data, a proxy server helps distribute requests without overwhelming the target site.

4. Making Requests to These URLs

Once URLs are identified and policies are checked, the next step is sending HTTP requests (GET requests, typically) to the server hosting those web pages. This is how the scraper retrieves the HTML content of the web pages.

5. Using Locators to Identify the Data

After receiving the HTML, the scraper uses specific selectors like CSS selectors, XPath, or other locators to identify where the required data is located on the page (e.g., specific tags like <div>, <span>, etc.).

6. Parsing the Data String

The scraped content, which is often in HTML, needs to be cleaned and parsed. This involves extracting the useful data from the raw HTML, converting it into a readable and structured format like text, JSON, or CSV.

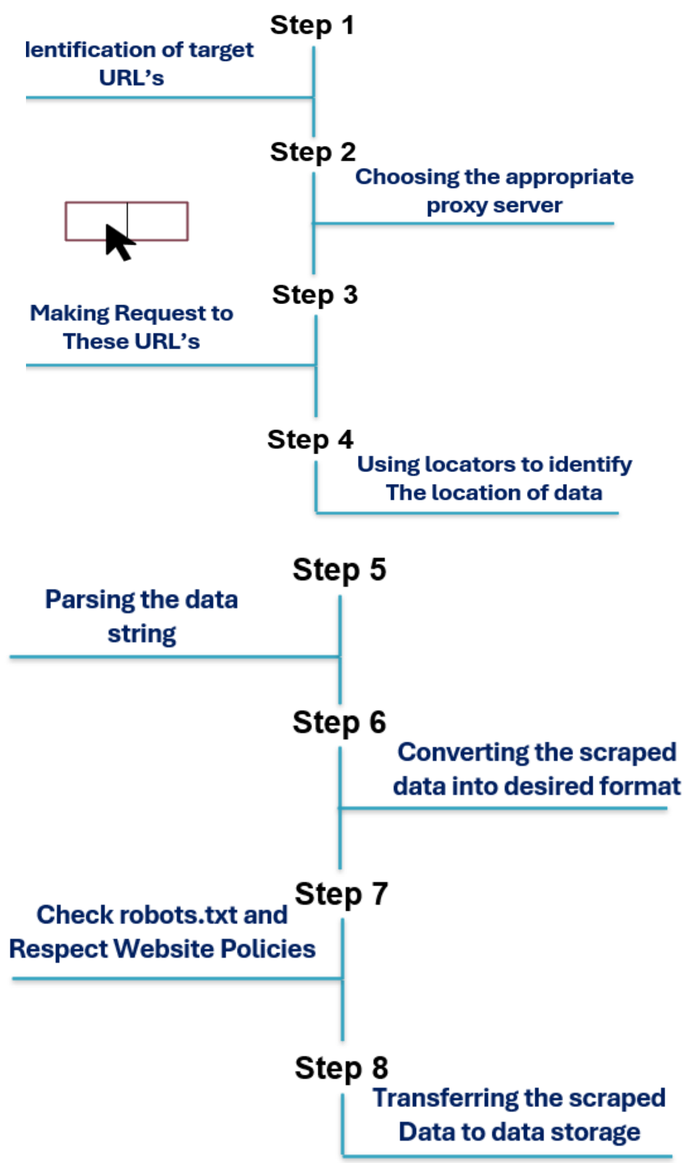
7. Converting the Scraped Data into Desired Format

The extracted data is often converted into a format suitable for analysis or storage. This could include transforming it into a spreadsheet format (like CSV or Excel), a database entry, or another structured form like JSON or XML.

8. Transferring the Scraped Data to Data Storage

Once the data is cleaned and converted, the final step is to save it in a database, a file (like CSV or Excel), or any storage system that will hold the structured data for future analysis or use.

This flow ensures that web scraping is performed systematically and efficiently maintaining legal and ethical guidelines.



COMMONLY USED TOOLS

- **Scrapy (Python framework)**:

  Scrapy is a powerful Python framework for web scraping, used primarily for creating spiders that crawl and scrape websites for data. It's highly efficient and can handle large-scale scraping projects, automating data collection tasks while managing requests, handling errors, and storing scraped data in various formats (JSON, CSV, etc.). Scrapy's strength lies in its scalability and support for managing requests through concurrency.

- **Beautiful Soup**:

  Beautiful Soup is a Python library that simplifies the process of scraping data from HTML and XML documents. It helps parse these documents into tree structures, making it easy to search and extract specific elements by their tags, classes, or IDs. It's ideal for small projects and one-off scraping tasks, allowing users to quickly grab and manipulate data without the need for complex setup.

- **ScrapingBee**:

  ScrapingBee is a service that simplifies web scraping by providing proxy management, browser simulation, and handling of JavaScript-rendered websites. It allows users to scrape data efficiently, even from websites that block bots, by rotating proxies and managing headers. ScrapingBee is well-suited for users who need reliable, scalable scraping with minimal configuration.

- **Selenium**:

  Selenium is a web automation tool that is primarily used for testing web applications but is also popular for web scraping. It allows users to interact with web elements, mimic human behavior, and automate the navigation of websites. Unlike other tools, Selenium handles JavaScript-heavy websites well by controlling a real browser, making it a powerful tool for scraping dynamic content that relies on JavaScript for rendering.

APPLICATIONS

1. Price Monitoring

Web Scraping can be used by companies to scrap the product data for their products and competing products as well to see how it impacts their pricing strategies. Companies can use this data to fix the optimal pricing for their products so that they can obtain maximum revenue.

2. Market Research

Web scraping can be used for market research by companies. High-quality web scraped data obtained in large volumes can be very helpful for companies in analyzing consumer trends and understanding which direction the company should move in the future.

3. News Monitoring

Web scraping news sites can provide detailed reports on the current news to a company. This is even more essential for companies that are frequently in the news or that depend on daily news for their day-to-day functioning. After all, news reports can make or break a company in a single day!

4. Sentiment Analysis

If companies want to understand the general sentiment for their products among their consumers, then Sentiment Analysis is a must. Companies can use web scraping to collect data from social media websites such as Facebook and Twitter as to what the general sentiment about their products is. This will help them in creating products that people desire and moving ahead of their competition.

5. Email Marketing

Companies can also use Web scraping for email marketing. They can collect Email ID’s from various sites using web scraping and then send bulk promotional and marketing Emails to all the people owning these Email ID’s.

CHALLENGES AND LIMITATIONS

Limitations of Web Scraping

- **Learning curve**:
  - Even easy tools take time to master, and coding skills like XPath and AJAX are often required.
- **Changing website structures**:
  - Website updates can break existing scrapers.
- **Handling complex websites**:
  - Features like AJAX and infinite scrolling complicate scraping.
- **Large-scale data extraction**:
  - Not all tools can handle millions of data records.
- **Limited data types**:
  - Text and URLs are easier to extract; PDFs and images are harder.
- **IP blocking**:
  - Websites may block scrapers, requiring CAPTCHA solutions and proxies.
- **Legal concerns**:
  - Scraping private data without permission can lead to legal trouble.

CONCLUSION

In conclusion, web scraping is a vital tool for efficiently gathering data from websites, facilitating sectors like market research, pricing surveillance, and academic studies. Despite its potential, web scraping poses challenges such as adapting to constantly evolving website structures and ensuring compliance with legal and ethical standards.

Moreover, technical difficulties like CAPTCHA systems and anti-scraping mechanisms require users to choose the right tools and strategies to manage these obstacles. Ensuring that the scraping process is ethical, legal, and efficient is crucial to leveraging web scraping's potential in data-driven decision-making.

Looking ahead, as the digital world evolves, so will the techniques and tools for web scraping. Future advancements in AI and machine learning may simplify the process further, enhancing accuracy and reducing technical barriers. Those who use web scraping responsibly will continue to unlock invaluable insights that contribute to innovation, research, and business growth.

REFERENCES

- Zhao, B. (2022). ***Web scraping***. In Encyclopedia of big data (pp. 951-953). Cham: Springer International Publishing.
- Singrodia, V., Mitra, A., & Paul, S. (2019, January). ***A review on web scrapping and its applications***. In 2019 international conference on computer communication and informatics (ICCCI) (pp. 1-6). IEEE.
- GeeksforGeeks. (n.d.)***. What is web scraping and how to use it?*** GeeksforGeeks.
- SDL Corporation. (2024, January 10). ***What are the top 10 web scraping tools recommended for 2024***? SDL Corporation. https://sdlccorp.com/post/what-are-the-top-10-web-scraping-tools-recommended-for-2024/
- ScraperAPI. (n.d.*)*. ***How does web scraping work***? ScraperAPI. https://www.scraperapi.com/web-scraping-learning-hub/how-does-web-scraping-work/